

CHAPTER 3

METHODOLOGY

This chapter describes the structured approach and systematic development lifecycle adopted to design and develop the proposed Internship E-Logbook System. The Waterfall Model has been selected as the software development methodology to achieve the project objectives in an organised and efficient manner. The Waterfall Model follows a sequential, linear approach in which each phase, namely Requirement Analysis, System Design, Implementation, Testing, Deployment and Documentation, must be fully completed before proceeding to the next. This methodology ensures clear documentation, strict control over project requirements, and structured milestones that directly support the development of the mobile application, web dashboard, database and AI-assisted features outlined in Chapters 1 and 2.

3.1 Project Framework (Software Development Methodology)

The Waterfall Model will be used as the software development methodology for developing the Internship E-Logbook System because it ensures that every step involved in its software development life cycle is carried out one after the other. The steps are requirement analysis, system design, implementation, testing, and deployment and documentation. This methodology is deemed appropriate for the current project since the requirements of the system in terms of features such as digital logbook entry, AI-powered writing suggestions, notification alerts, and supervision have been determined in the background study and literature reviews carried out in Chapters 1 and 2, hence making the possibility of any drastic changes during the process of development unlikely. Related studies such as Surahmat and Darmawan (2024) and Dewi and Hanifah (2024) who used SDLC methodology together with the Laravel framework to develop web-based information system and web-based internship monitoring system respectively indicated that methodologies like these make the process of development systematic and easily implementable. The Waterfall Model makes it easier for the project supervisor to follow and monitor the development process due to the systematic stages and documentation developed in each stage making it feasible for a single developer FYP project to be developed in a certain period of time in an academic context.

However, despite the fact that several external cloud services the Gemini API, Pusher Channels, Firebase Cloud Messaging, Cloudinary and Neon have been included in the project, whose behaviour and documentation in free tiers cannot be fully managed by the developer, an Agile or Incremental approach will not be applied to this project due to the fact that the connection points with all those external services are specified and well separated into certain modules that will be determined in the Requirement Analysis phase (Section 3.2) AI-Assisted Writing, Real-Time Chat, Notification, and Image Documentation modules instead of being scattered all over the system arbitrarily. Thus, any changes associated with the third-party API behaviour are expected to be restricted to the respective modules and can be implemented in the Implementation and Testing phases without going back to the requirements and system design formulated in the previous phases. However, this approach to building the application is considered as a limitation of applying the Waterfall Model in this case.

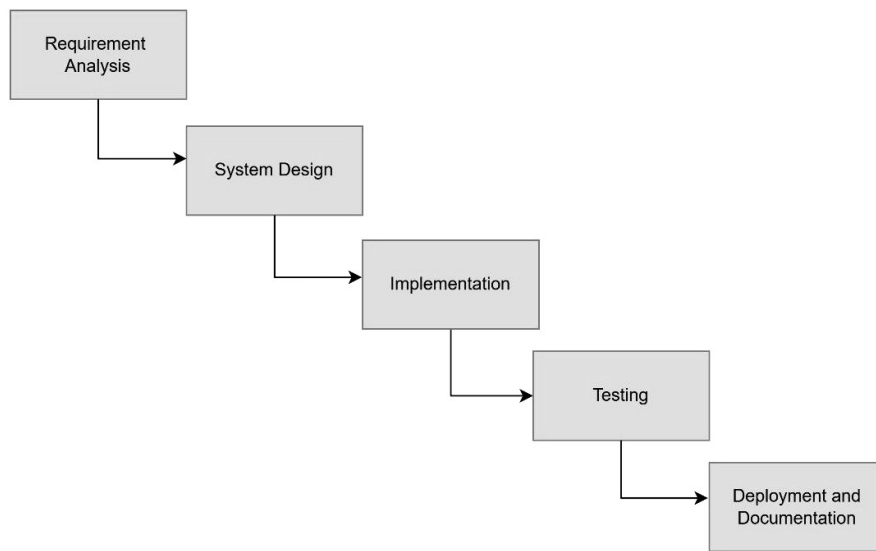


Figure 3.1 Waterfall Model Framework

As illustrated in Figure 3.1, the Waterfall Model adopted for this project consists of six sequential phases: Requirement Analysis, System Design, Implementation, Testing, Deployment and Documentation, with the output of each phase serving as the input to the next.

3.2 Phase 1: Requirement Analysis

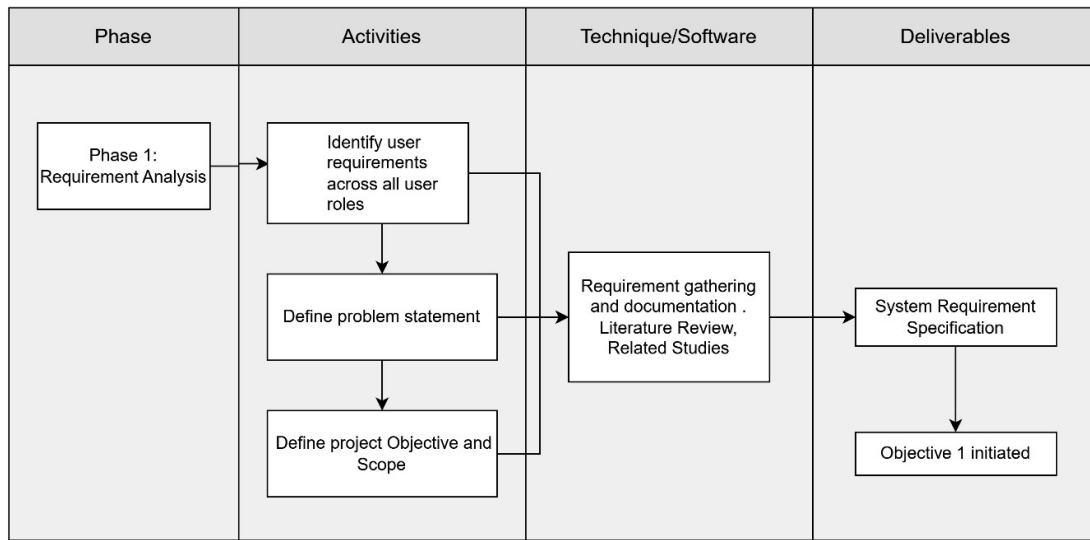


Figure 3.2 Requirement Analysis

As illustrated in Figure 3.2, the Requirement Analysis stage will start with a preliminary analysis in order to create an overview of the problem domain that is to be solved prior to any design and development activities. A preliminary analysis is to be done via background study and literature analysis, which are described in Chapter 1 and Chapter 2, not by means of a survey, interview, or questionnaire because the potential target respondents (the internship students, the lecturers, and the company’s supervisors) are not available for research during the project period of time. Literature analysis of four intern management and monitoring systems (Ahmed et al., 2026; Wan Abdul Rahman et al., 2024; Castro, 2024; Dewi & Hanifah, 2024) reveals some recurrent shortcomings of the systems, such as inability to access the system from mobile devices, no possibility of monitoring process in real time, no help in writing by means of artificial intelligence and automatic reminders, etc. All of these factors and the problem statement in Section 1.2 will be used to derive the system requirements.

The functional requirements of the system that have been recognized include user authentication and role-based access, digital logbook entries generation and submission, AI-powered writing advice, documentation uploading by means of images, live messaging between students and mentors, as well as push notification reminders. The non-functional requirements comprise data security using token authentication, system availability through cloud-hosting, and appropriate response time for live functions such as chat and notifications.

Based on these requirements, the hardware and software needed to develop the prototype will be identified, as summarised in Tables 3.1 and 3.2.

Table 3.1 Hardware Requirement

Hardware	Specification	Purpose
Laptop	Windows 10/11, minimum 8 GB RAM, Intel Core i5 or equivalent	Used for coding, running emulators, and managing the backend server during development
Android smartphone	Android 8.0 or above	Used for testing the Flutter mobile application on a physical device
Internet connection	Minimum 10 Mbps	Required to access cloud-hosted services (Neon, Pusher, Cloudinary, Gemini API, Firebase) during development and testing

Table 3.2 Software Requirements

Software/Tool	Purpose
Flutter SDK and Dart	Development of the cross-platform mobile application
Visual Studio Code	Primary code editor for both the Flutter application and Laravel backend
Laravel Framework (PHP)	Development of the backend RESTful API and web-based administrative dashboard
PostgreSQL (hosted on Neon)	Storage and management of structured application data
Google AI Studio (Gemini 1.5 Flash)	Provision of AI-assisted writing suggestions within the logbook module
Pusher Channels	Real-time chat messaging between students, lecturers, and company supervisors
Firebase Cloud Messaging	Delivery of push notifications for reminders and chat alerts
Cloudinary	Storage and delivery of internship documentation images

3.3 Phase 2: System Design

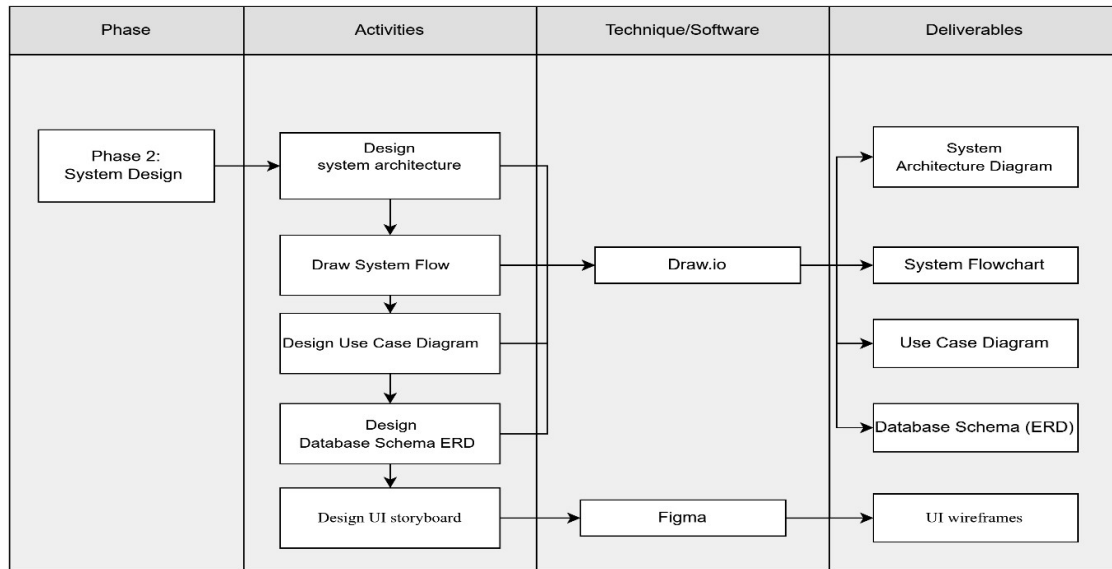


Figure 3.3 System Design Phase

As illustrated in Figure 3.3, The System Design phase will translate the requirements identified in Phase 1 into a structured technical design, covering the overall system architecture, process flow, use case diagram, system architecture, database structure and user interface layout of the proposed system.

3.3.1 System Flowchart

The system flowchart will illustrate the overall process flow of the Internship E-Logbook System, beginning from user login and role verification, followed by role-specific functions such as logbook entry creation, AI-assisted writing, supervisor review, chat communication, and notification delivery.

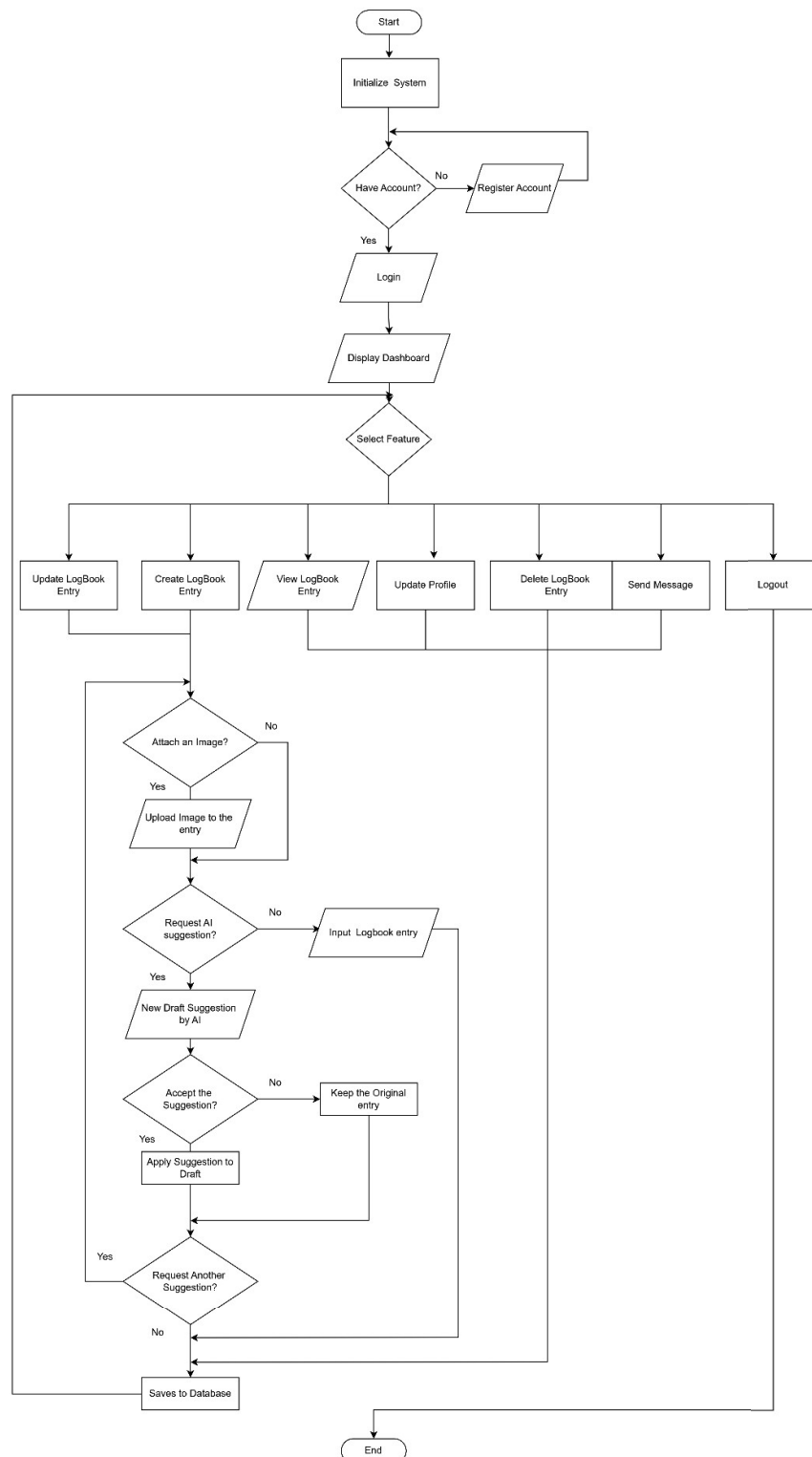


Figure 3.4 Student Flowchart

Figure 3.4 shows the process flow of the E-Logbook Mobile Application, starting from the system initiation and the authentication of users. After initiating the application, the user is

redirected to the dashboard page from where he/she can perform multiple tasks like creating, viewing, updating or deleting logbook entry, updating his/her profile, sending messages, and finally logging out of the account. If the user is going to create a logbook entry, then he/she can upload image(s) in support and can seek the help of AI for suggestions regarding writing of content and can either accept or reject the content suggested by AI. Finally, after making an entry, it will be stored in the database.

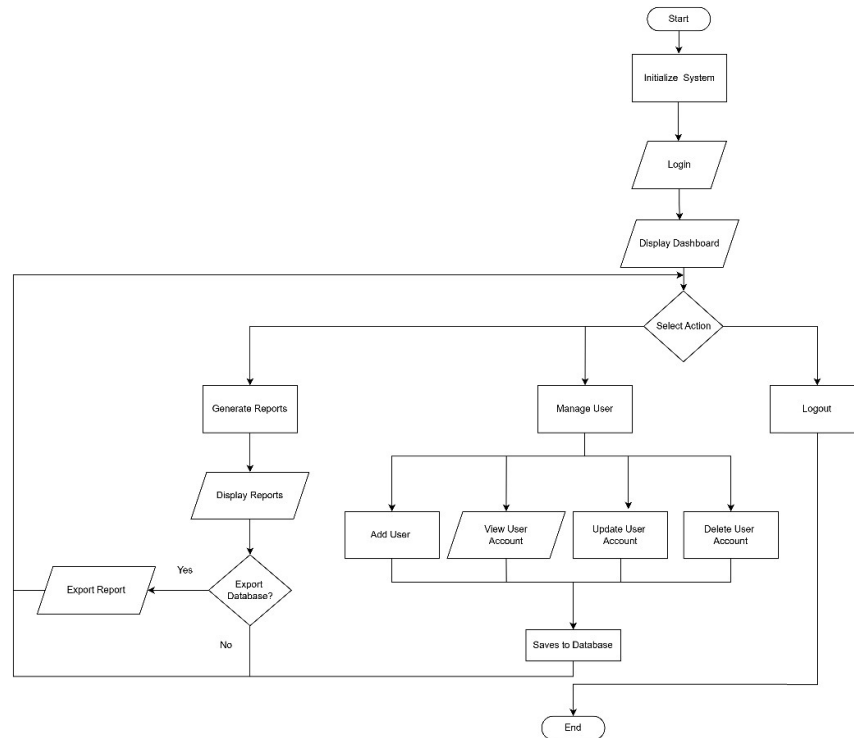


Figure 3.5. Admin Flowchart

Figure 3.5 describes the complete workflow of the user management and report generation system. First of all there is system initialization after which user logs into the system, and the dashboard appears on the screen. User selects an activity after accessing the dashboard, where he/she may manage the users, report generation, or logout from the system. In case of user management module, administrator performs various activities like addition of new users, viewing details of existing users, updating details of existing users, and deletion of users' account in the database. In case of reporting module, the user generates the report and displays it on the screen with the choice of exporting database or reports wherever necessary.

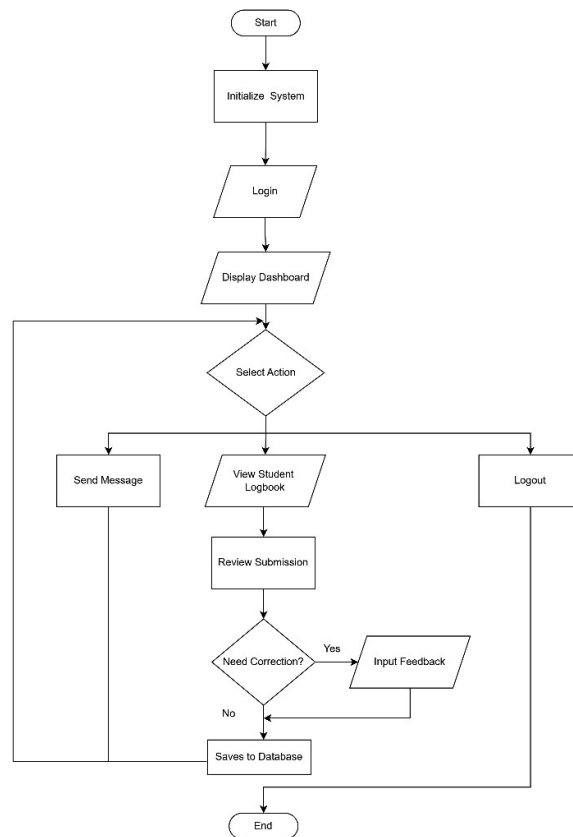


Figure 3.6. Lecturer and Company Supervisor Flowchart

Figure 3.6 shows the working process of the lecturer module of the student logbook management system. The working process starts from system initialization, then there will be a step for user login and dashboard will appear. After dashboard appears, lecturer performs actions such as checking student logbook, sending messages or logout. In the case where lecturer is checking student logbook, lecturer checks the works submitted by students and makes decisions whether corrections have to be made or not. If corrections have to be made, lecturer writes feedback for students' works. All the checks and feedbacks are stored in the database and then the process comes to an end.

3.3.2 System Architecture

Architecture used in the system is that of three-tier approach, including presentation tier, application tier, and data tier along with cloud services for the functioning of the system. As shown in Figure 3.7, the architecture shows how Flutter mobile application and Laravel web dashboard interact with Laravel backend that acts as the core of business logic and data

handling processes of the system. In addition to the backend, the system will integrate with different cloud services such as Neon PostgreSQL, Gemini API, Pusher Channels, FCM, and Cloudinary.

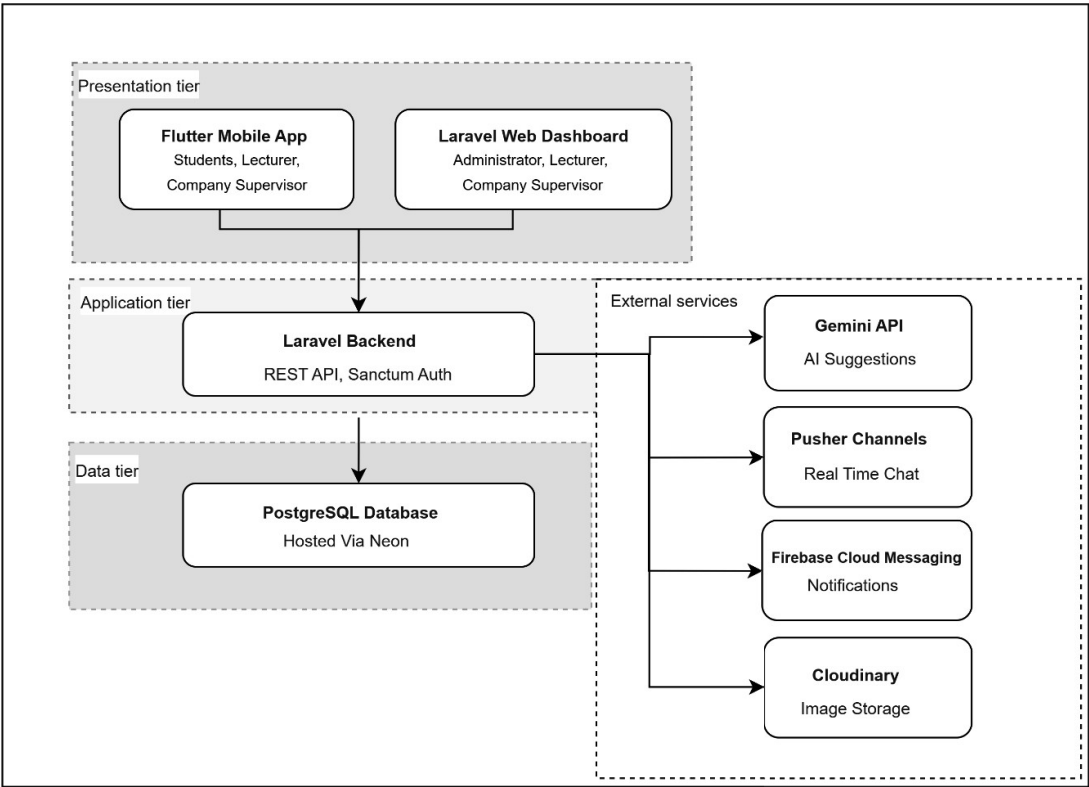


Figure 3.7. Three-Layer System Architecture

The proposed architecture implements a three-tier architecture that involves presentation tier, application tier, and data tier, along with the integration of a number of cloud-based services. In the presentation tier, there is interaction between users and the system via two different interfaces depending on user types. In this case, the mobile application developed in Flutter allows students, lecturers, and company supervisors to use various features related to internships such as submitting reports, tracking progress, communicating, and receiving notifications using mobile devices. Additionally, the Laravel web-based dashboard is available for administrators, lecturers, and company supervisors to perform various tasks in a browser-based manner.

The Laravel backend is the key component in the application layer as it deals with all the requests coming from the client applications. It contains the implementation of the business logic, verifies the data provided by users, takes care of the authentication process via Laravel

Sanctum, and communicates with the database and other cloud services. Instead of making the client applications interact directly with the third-party services, all of the requests go through the backend, which gives an extra layer of protection by securing API credentials while keeping everything under one roof. The backend also works together with the Gemini API that provides AI assistance for writing, Pusher Channels that help establish real-time communication, Firebase Cloud Messaging (FCM) that sends push notifications, and Cloudinary for images upload.

The data layer is made up of a PostgreSQL database that is hosted in Neon. The database is used as the primary data storage for everything in the application including users' account details, internships, reports, attendances, chats, notifications and other information. Through the use of a database that is hosted in the cloud, there will be an increase in the scalability, reliability, and maintenance of the system while at the same time keeping data secure and accessible. In general, the layered architecture enables the separation of the user interface, application layer and data layer from each other.

3.3.3 Use Case Diagram

The use case diagram (Figure 3.8) illustrates the functional requirements of the proposed E-Logbook Mobile Application with AI-Assisted Writing and Reminder System by identifying the interactions between the system and its four primary actors: Student, Lecturer, Company Supervisor, and Administrator. Each actor is assigned specific responsibilities and system functionalities according to their respective roles, ensuring that users can only access features relevant to their permissions.



Figure 3.8. System Use Case Diagram

The students are the main users of the system who are able to carry out actions like login, edit profile, create, view, update and delete logbook entries, upload attachments, request AI writing assistance, view supervisor feedback, send messages, and receive notifications. The lecturers and company supervisors are tasked with the role of overseeing the internship process of the students via evaluation of logbooks, giving feedback, interacting with the students using the messaging feature and getting notifications about internship processes. The administrator is tasked with management of user accounts, supervision of the system activities using the dashboard, generation of reports and maintenance of the overall system process.

Generally, the use case diagram helps in illustrating the functional behavior of the system through describing the interactions between the users and the system.

3.3.4 Entity-Relationship Diagram (ERD)

The ERD (Figure 3.9) represents the database structure of the system, showing the key entities and their relationships, including Users, Logbook, Attachment, Message, Notification, and Feedback. Each logbook entry is linked to a single student and may contain multiple attachments and feedback entries, while each user is assigned a role that determines their level of access within the system.

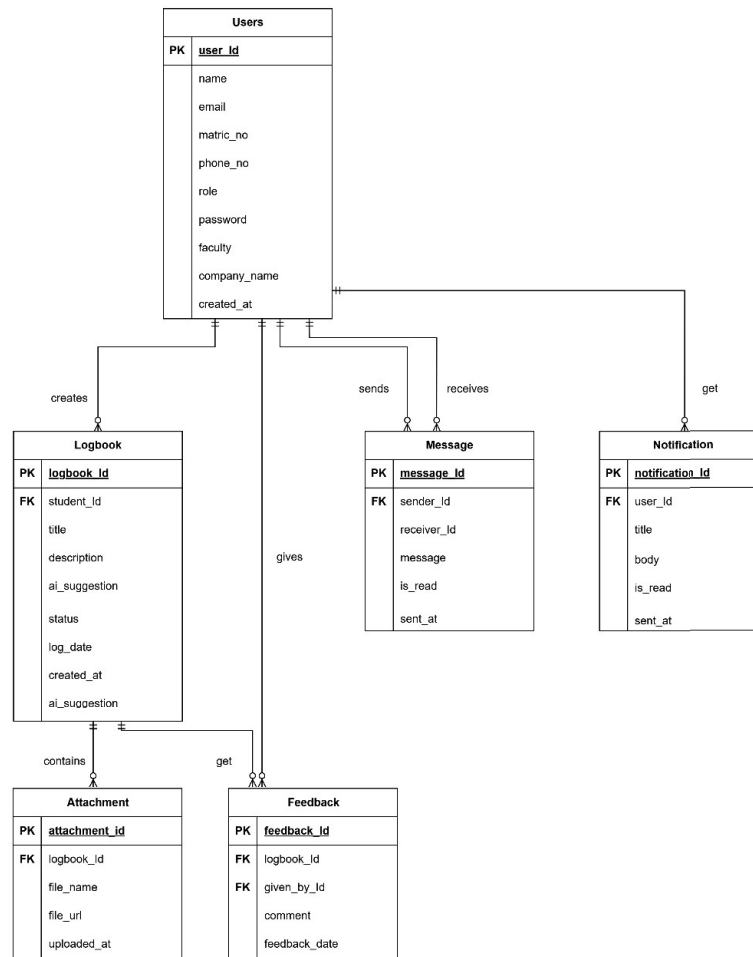


Figure 3.9. System ERD

The entity named Users holds the data for all the system users such as students, lecturers, supervisors from companies, and admins. It stores the profile data, log-in information, and roles of the users.

The Logbook object keeps track of the activities of each student involved in an internship, and also tracks their daily progress during the duration of the internship

The Message object controls communication among students, lecturers, and company supervisors. The message object stores all the chat messages that take place within the system.

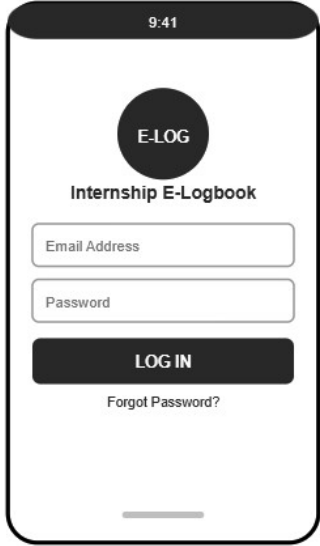
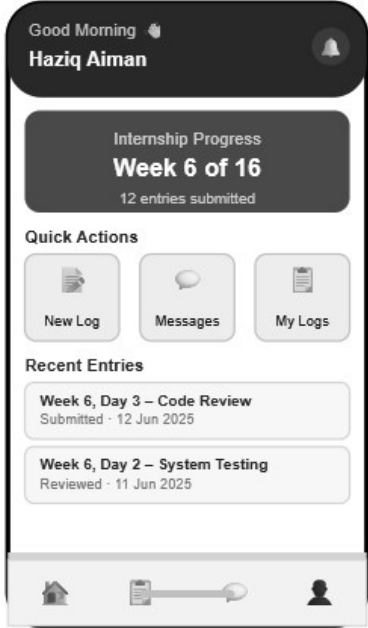
The Notification entity holds system notifications generated and sent to the users regarding certain important events, such as new message, feedback, reminder, logbook update, and other activities associated with the internship program.

The Attachment entity is an entity that contains links to files that the students upload as proof of their logbook activity. These attachments could be images, documents, and other files that are linked to the respective logbook record.

The Feedback entity holds the comments or reviews written by the lecturers/supervisors regarding the logbook entry submitted by the students.

3.3.5 Storyboard and User Interface Design

The storyboard illustrates the sequence of screens and user interactions in the proposed system. It outlines the navigation flow to ensure the user interface is intuitive, consistent, and easy to use. It also provides a visual overview of the system's main features and helps verify that users can complete their tasks efficiently throughout the application.

Storyboard	Description
 Figure 3.10. Login Page	This page allows users to log in using their registered email address and password. After successful authentication, users are redirected to their respective dashboard based on their assigned role.
 Figure 3.11. Student Dashboard	This page provides students with an overview of their internship progress and quick access to key features such as creating logbook entries, viewing logbooks, and accessing the messaging interface.

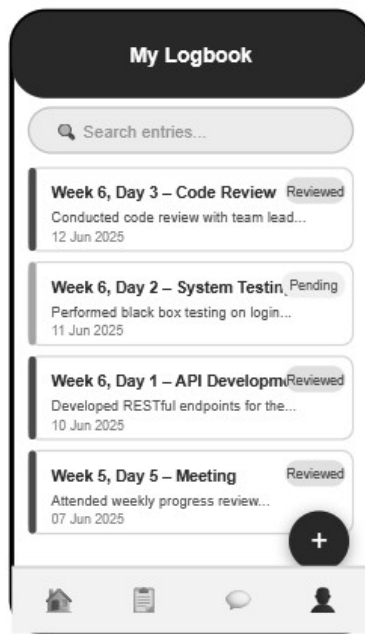


Figure 3.12. Logbook List Page

This page displays all logbook entries submitted by the student. Users can search for specific entries, view their submission status, and create new logbook entries.

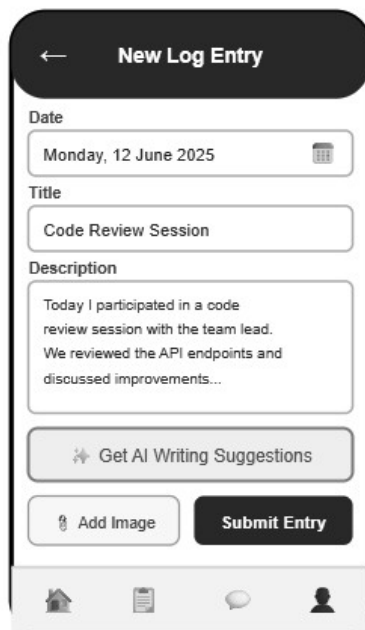
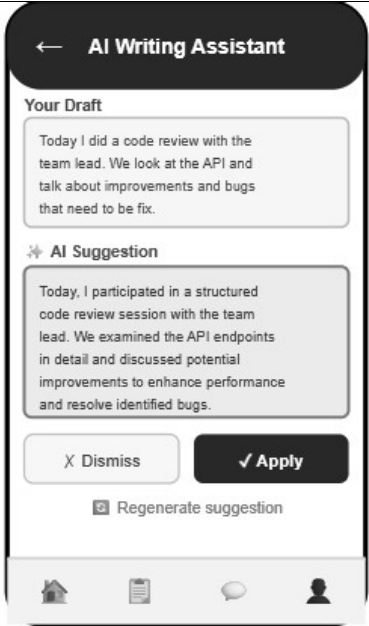


Figure 3.13. Create Logbook Entry Page

This page enables students to create and submit new logbook entries by entering internship activities, attaching supporting images, and requesting AI-assisted writing suggestions before submission.

 The screenshot shows the 'AI Writing Assistant' interface. At the top is a back arrow and the title 'AI Writing Assistant'. Below is a section titled 'Your Draft' containing a text box with the text: 'Today I did a code review with the team lead. We look at the API and talk about improvements and bugs that need to be fix.' Below this is an 'AI Suggestion' section with a text box containing: 'Today, I participated in a structured code review session with the team lead. We examined the API endpoints in detail and discussed potential improvements to enhance performance and resolve identified bugs.' At the bottom of the suggestion box are two buttons: 'X Dismiss' and '✓ Apply'. Below these buttons is a link that says '🔄 Regenerate suggestion'. At the very bottom is a navigation bar with four icons: a house, a document, a speech bubble, and a person.	This page displays AI-generated writing suggestions to help students improve their logbook content. Students may apply, discard, or regenerate the suggested text.
 The screenshot shows the 'Upload Evidence' interface. At the top is a back arrow and the title 'Upload Evidence'. Below is a dashed box containing a camera icon and the text 'Tap to take photo or select from gallery'. Below this is a section titled 'Uploaded (2/5)' showing two image thumbnails. The first is labeled 'workspace.jpg' and the second is labeled 'meeting.jpg'. Each thumbnail has a close button (X) in the top right corner. Below the thumbnails is the text 'Max 5 images · JPG/PNG · Max 5 MB each'. At the bottom is a large button labeled '✓ Done'. At the very bottom is a navigation bar with four icons: a house, a document, a speech bubble, and a person.	This page allows students to upload supporting images for their logbook entries by capturing a photo or selecting one from the device gallery before submission.

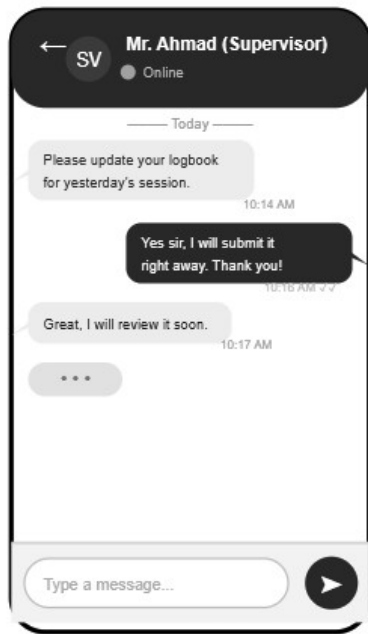


Figure 3.16. Messaging Page

This page enables real-time communication between students, lecturers, and company supervisors, allowing users to exchange messages and receive updates during the internship.

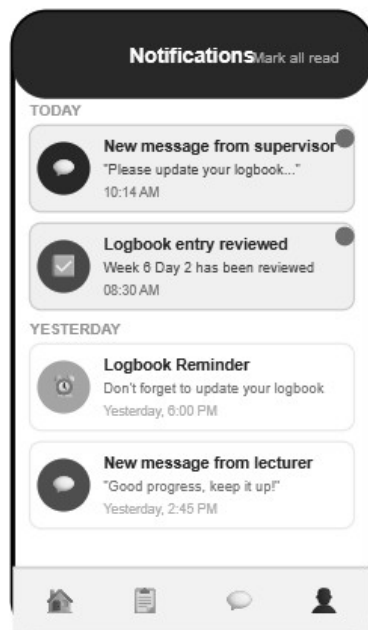


Figure 3.17. Notification Page

This page displays system notifications, including reminders, logbook updates, feedback, and new messages, ensuring users remain informed of important activities.

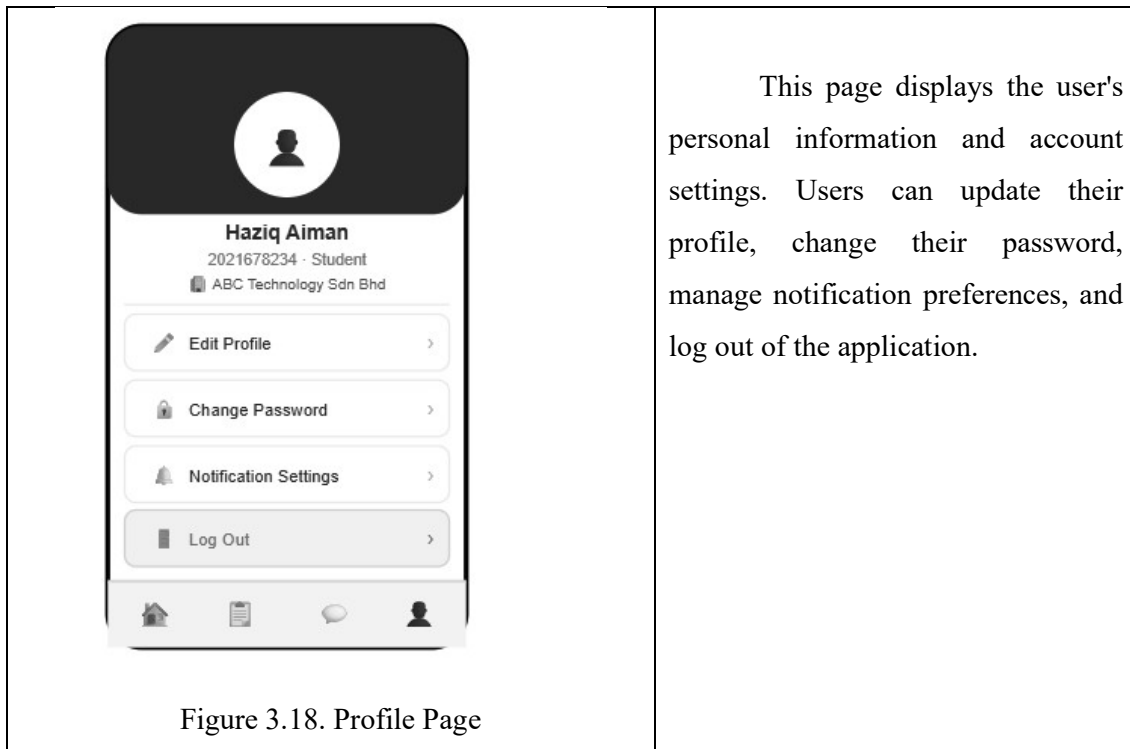


Figure 3.18. Profile Page

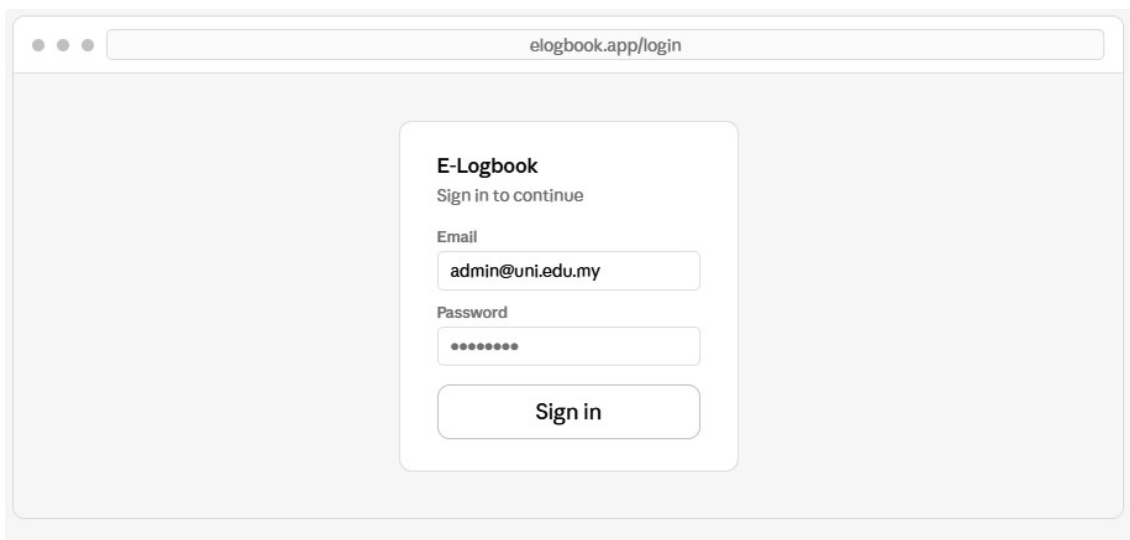


Figure 3.19. Webpage Login page

Entry point for admins and supervisors. Users enter their registered email and password. Laravel Sanctum verifies credentials and redirects each role to the appropriate dashboard view.

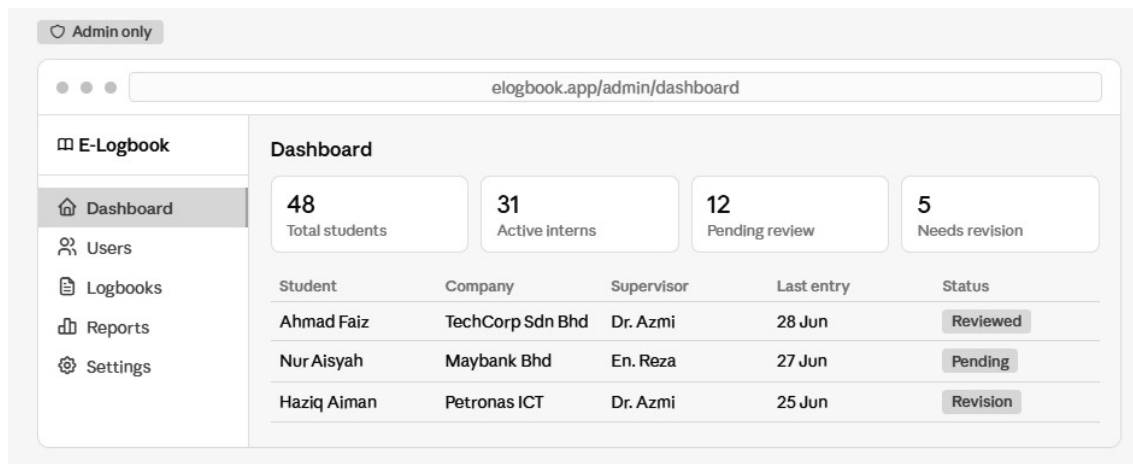


Figure 3.20. Admin Dashboard

This page provides administrators with an overview of the system, including internship statistics, user information, and pending logbook submissions. It also offers quick access to user management and report generation features.

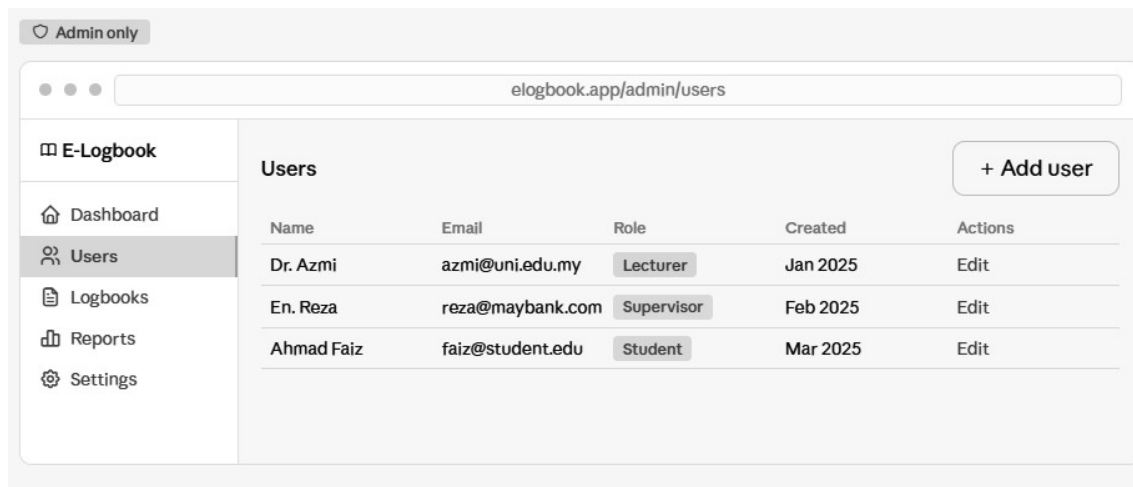


Figure 3.21. Admin User Management

Administrators have the ability to manage all users' accounts on this page, from the students to the lecturers and company supervisors. They have the ability to create, update, delete users' accounts and assign roles to them.

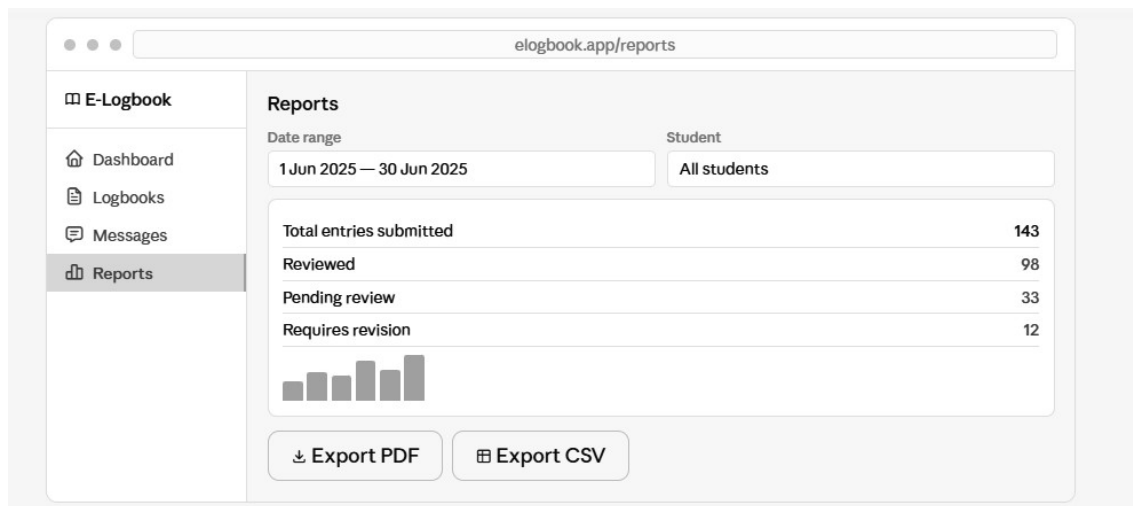


Figure 3.22. Generate and Download Reports

Admins can generate system-wide reports across all students. Lecturers and supervisors generate reports scoped to their own assigned students only.

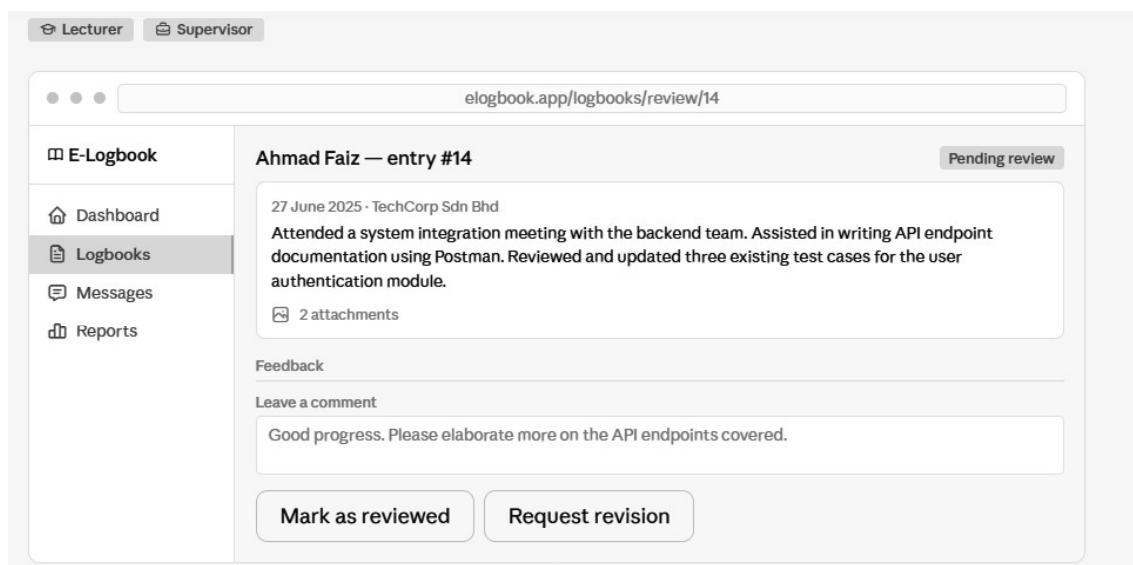


Figure 3.23. Logbook Review

View student entries and give feedback. The lecturers and company supervisors get to see all of their assigned student logbook entries in great detail. They can choose to either review, request revision, or give feedback on the entry.

3.4 Phase 3: Implementation

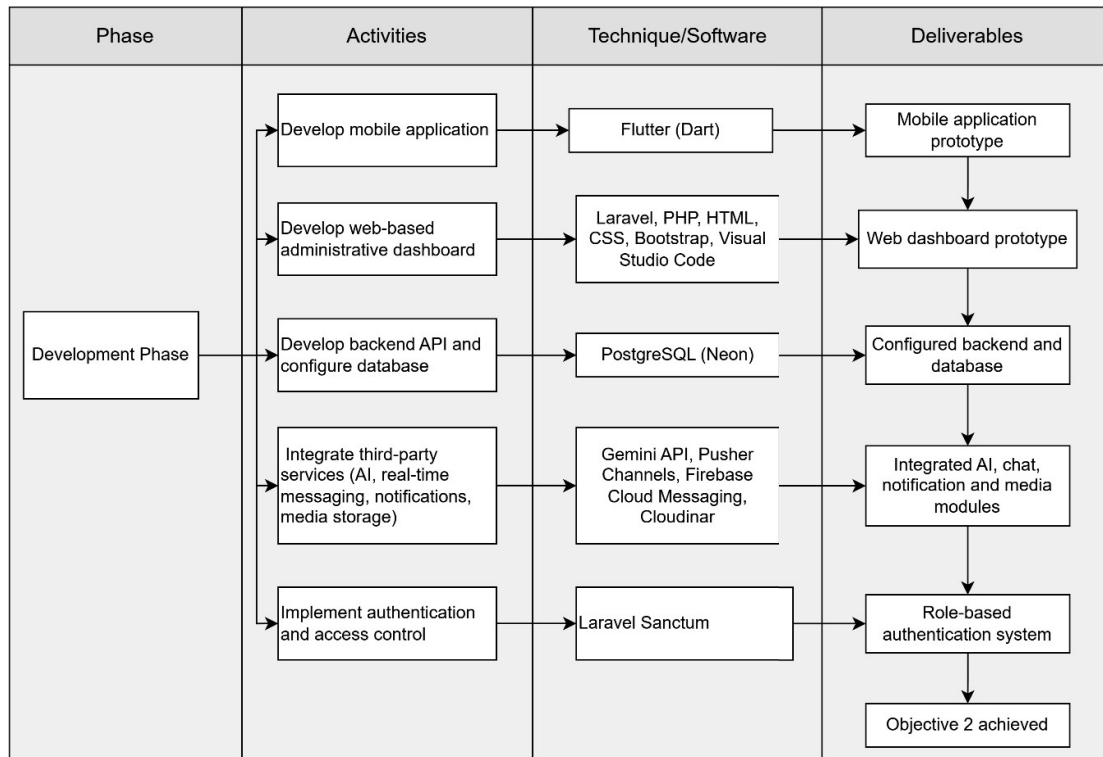


Figure 3.24 Implementation Phase

The Implementation phase will involve translating the system design produced in Phase 2 into a functional prototype. Development will follow the sequence of activities illustrated in Figure 3.24, beginning with the development of the mobile application and web-based administrative dashboard, followed by the implementation of the backend API and database, integration of third-party services, and finally the implementation of authentication and access control. Each component will be developed using the technologies justified in Section 2.4, resulting in a fully integrated internship logbook management system.

3.4.1 Mobile Application Development

This mobile application will be developed using the Flutter software technology together with the Dart programming language. This application is the main interface that will be used by students to keep track of their logbooks, communication with supervisors, get notifications, and use AI-based writing tools. Lecturers and company supervisors can also

make use of this mobile application when reviewing logbooks and giving feedback to students without the use of a desktop computer.

3.4.2 Web-Based Administrative Dashboard Development

The dashboard that is accessible through the Web will be created using Laravel, PHP, HTML, CSS, Bootstrap, and Visual Studio Code. This dashboard will enable the administrators to manage the systems, including user management, system monitoring, and reporting. Also, lecturers and company supervisors will use this dashboard to assess the submission of logbooks, give comments on it, and create reports of the assigned students. The development process is expected to result in a working web dashboard prototype.

3.4.3 Backend API and Database Development

For building the backend API, the Laravel framework will be utilized, and PostgreSQL running on Neon will be employed as the database for the system. The Laravel Eloquent ORM will be used for handling communication between the database and the application. The backend will offer RESTful API endpoints for user authentication, logbook operations, reporting, and other functionalities of the system, while the database will store user credentials, logbooks, feedback, notifications, and other system data.

3.4.4 Third-Party Service Integration

Integration of some other third-party systems will be performed to augment the system's capabilities. The Gemini API will be integrated using the Laravel backend to offer assistance with writing suggestions that include both text only and text image requests using AI support. Pusher Channels will be integrated using Laravel Broadcasting and Flutter to allow real time messaging between students and supervisors. Firebase Cloud Messaging will be used to send push notifications for reminders, feedback and chat messages, whereas Cloudinary will be integrated to save logbook images uploaded by users and to retrieve them back using CDNs via their URLs.

3.4.5 Authentication and Access Control Implementation

Role-based authentication will be achieved by using Laravel Sanctum for token-based authentication in combination with the Spatie Laravel Permission package. After logging in, if the user's credentials are authenticated by the backend, the backend will generate a personal access token that will be securely stored using the package in the Flutter application. The access permissions will depend on the user's role. The students will have access to the application only via the mobile app, whereas the lecturers and company supervisors will have access to the mobile app and the web dashboard, but the administrator will have access only to the web dashboard.

3.5 Phase 4: Testing

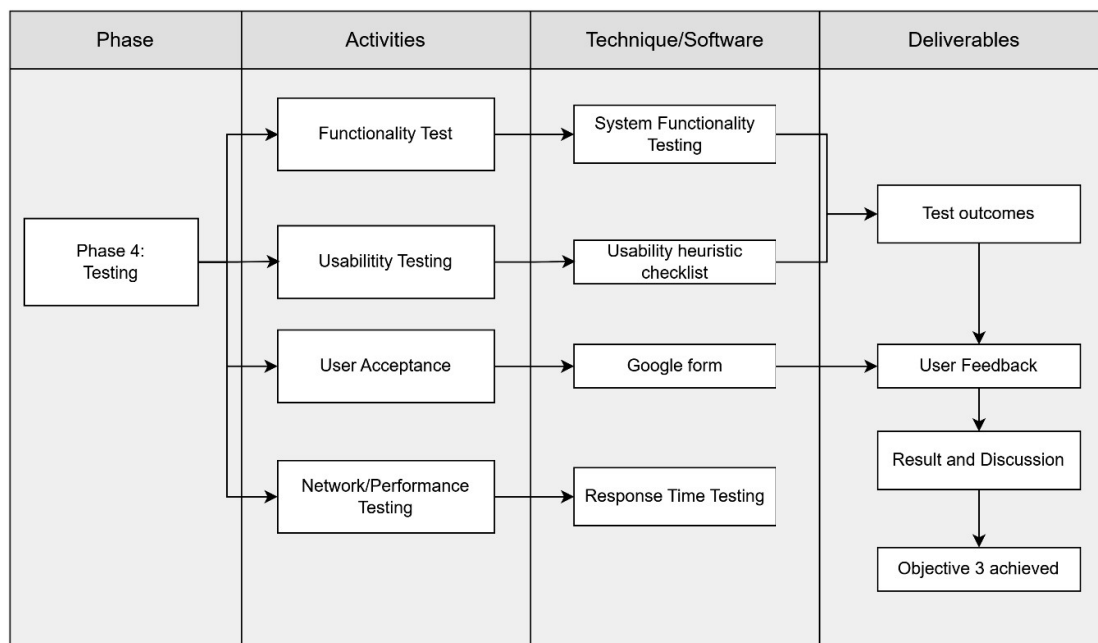


Figure 3.25 Testing Phase

As illustrated in Figure 3.25, in the Testing stage, it will be ensured that each module of the prototype works correctly and provides an adequate user experience through four forms of testing including functional, usability, user acceptance testing (UAT) and network performance testing. In functional testing, the technique of black box testing will be applied where the system will be tested on the basis of its input-output behavior without looking into

the source code of the system. It is believed that this technique is most appropriate to verify each feature's correctness from users' point of view.

Tests would be designed that would focus on the primary functionality of each module designed for the application in the system design phase. In this regard, the tests will be recorded along with the modules being tested, the testing process involved, and the output desired. For instance, the authentication module will be tested using valid and invalid login attempts to check for proper access control, the logbook module through the addition of new entries in the system to ensure the information is stored and displayed correctly, the AI-based writing tool through verifying the generation of recommendations in a reasonable time frame, the chat module by ensuring the transmission of the message to the other user in real time, the notification module by testing the receiving of messages when the users are offline, and the image documentation process through the upload of supportive images.

In addition to the functional correctness test cases, usability tests will be carried out through observation of representative users in each of the four roles: student, lecturer, company supervisor, and administrator, as they perform tasks on the mobile application and web dashboard, in order to detect any possible problems with the user interface or navigation. The user acceptance test will be done through actual usage of the system by the students and the supervisors and will provide feedback on whether the system serves the purpose of managing internship logbooks, in line with Objective 3. The performance of the system will also be tested based on its behavior in various network environments, due to the dependence of the system on external services such as Gemini API, Pusher Channels, Firebase Cloud Messaging, and Cloudinary, all of which need a good network connection to work effectively.

The test cases that result in failure will be recorded together with the cause of the problem, and the module revised and retested until the expected output is obtained. This will continue until all the requirements established during the requirement analysis are tested successfully.

3.6 Phase 5: Deployment and Documentation

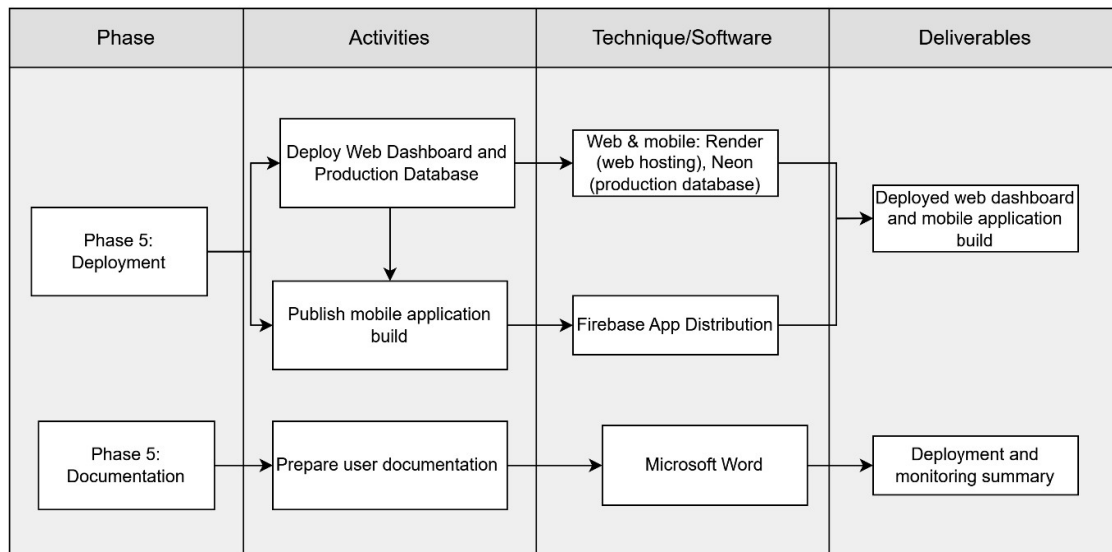


Figure 3.26 Deployment and Documentation Phase

As illustrated in Figure 3.26, the Deployment phase concentrates on the deployment of the completed Internship E-Logbook System to make it operational. At the Deployment phase, the Laravel web dashboard will be deployed in Render, which is a cloud hosting service provider, while the production PostgreSQL database hosted at Neon will be properly set up to provide a secure and reliable storage for the application data. The Flutter mobile application will be developed as an Android APK package and delivered to the project supervisor and testers by means of Firebase App Distribution, thus being installed directly on the device without publishing to the Google Play Store. Also, integrated cloud services like FCM (Firebase Cloud Messaging) will be set up for push notifications functionality in the production environment. Upon completion of the Deployment phase, the basic performance of the system will be tested using the built-in dashboard at Render in order to assure the proper working of all key features such as authentication, logbook, AI-based writing, messaging, notifications, and images upload functionalities. Completion of the Deployment phase will result in the deployment of the web dashboard and mobile application build, as well as in the deployment and monitoring summary report.

The Documentation stage entails preparing all required documents to help users and facilitate any future system maintenance. The preparation of user manuals and other training documents will be done through Microsoft Word so that the students, lecturers, company

supervisors, and administration can use the system easily. Some of the things to be covered include the process of logging in, managing the logbook, messaging, notifications, reports creation, and managing one's account on the system. Feedback from users will also be sought during the first use of the system so that there can be improvements for the future. The deliverable for this stage is a comprehensive user manual.

3.7 Summary of the Chapter

This chapter illustrates the process followed to develop the proposed Internship E-Logbook System using the Waterfall Model. This chapter discusses the various phases involved in software development which include Requirement Analysis, System Design, Implementation, Testing, and Deployment and Documentation as well as the respective processes involved in each phase. In the Requirement Analysis phase, system requirements will be derived from the background study and literature review. System Design phase will convert the identified system requirements to system architecture, flow charts, use case diagram, entity relationship diagram, and user interface storyboard. Implementation phase will involve development of the Flutter mobile application, Laravel web dashboard and PostgreSQL database as well as integration of cloud services such as Gemini API, Pusher Channels, Firebase Cloud Messaging, Cloudinary, and Laravel Sanctum. Testing phase will involve verification that the system developed satisfies its functional and non-functional requirements through functional testing, usability testing, user acceptance testing, and network performance testing.

Every stage of the Waterfall Methodology corresponds directly with the objectives defined in section 1.3. The Requirement Analysis stage (Stage 1) is achieved through objective 1, in which the problems and constraints of manual logbooks management will be recognized. System Design and Implementation stages (Stages 2 and 3) are accomplished through objective 2, in which the development of the system with the use of artificial intelligence on mobile devices will be carried out. Finally, the Testing stage (Stage 4) is achieved through objective 3, in which the efficiency of the developed system will be assessed. Thus, the methodology gives a framework that ensures the achievement of project objectives, laying the basis for the implementation and assessment results to be described in the next chapter.

REFERENCES

- Ahmed, M. O., Neal, A. O., & Chizoba, C. (2026). A digital solution for student internship: Tracking and enhancing student's experience. *Asian Journal of Basic Science & Research*, 8(1), 23–36. <https://doi.org/10.38177/ajbsr.2026.8103>
- Akhtar, A. (2023). Popularity ranking of database management systems. *arXiv*. <https://arxiv.org/abs/2301.00847>
- Ambara, M. P., Gunawan, I. M. A. O., & Wijaya, I. W. R. (2024). Agile-based field internship information system for academic administration digitalization. *Jurnal Teknologi Informasi dan Pendidikan*, 19(1), 1–10. <https://doi.org/10.24036/jtip.v19i1.1069>
- Bawica, I. M. (2021). The university internship program and its effects on students' employability readiness. *International Journal of Academe and Industry Research*, 2(3), 90–106. <https://doi.org/10.53378/348731>
- Castro, E. G. M. (2024). Mobile-based student internship monitoring system using progress tracking algorithm. *International Research Journal on Advanced Science Hub*, 6(8), 204–209. <https://doi.org/10.47392/IRJASH.2024.029>
- Dewi, F. K., & Hanifah, R. N. (2024). Web-based student internship monitoring information system (Case study: SMK Ma'arif NU 2 Boyolali). *IC-ITECHS*, 5(1), 943–956. <https://doi.org/10.32664/ic-itech.v5i1.1705>
- Hartono, R. (2024). Website-based information system for internship attendance and daily activity recording. *Journal of Information Systems and Business Technology*, 3(1), 1–10. <https://journal.jci.co.id/jisbt/article/view/302>
- Haw, S.-C., Amin, A., Naveen, P., & Ng, K.-W. (2022). Performance evaluation of XML dynamic labeling schemes on relational database. *International Journal of Technology*, 13(5), 1055–1063. <https://doi.org/10.14716/ijtech.v13i5.5871>
- Joiner, D., Clement, M., Chan, S. T., & Pereira, K. (2022). DW vs OLTP performance optimization in the cloud on PostgreSQL: A case study. *IEEE RASSE 2022*. <https://doi.org/10.1109/RASSE54974.2022.9989603>
- Kapoh, H., Melo, O. E., & Kimbal, A. A. (2024). Digitalization of the internship management system to support independent learning campus program. *International Journal of Computer Applications*, 186(1), 16–21. <https://doi.org/10.5120/ijca2024923335>
- Kishore, K., Khare, S., Uniyal, V., & Verma, S. (2022). Performance and stability comparison of React and Flutter: Cross-platform application development. *IEEE ICCR 2022*.

<https://doi.org/10.1109/ICCR56254.2022.9996039>

- Lang, G., Triantoro, T., & Sharp, J. H. (2024). Large language models as AI-powered educational assistants: Comparing GPT-4 and Gemini for writing teaching cases. *Journal of Information Systems Education*, 35(3), 390–407. <https://doi.org/10.62273/YCIJ6454>
- Li, H., Wang, X., Feng, Y., Qi, Y., & Tian, J. (2024). Driving intelligent IoT monitoring and control through cloud computing and machine learning. *arXiv*. <https://doi.org/10.48550/arXiv.2403.18100>
- Mah, D.-K., & Groß, N. (2024). Artificial intelligence in higher education: Exploring faculty use, self-efficacy, distinct profiles, and professional development needs. *International Journal of Educational Technology in Higher Education*, 21, Article 58. <https://doi.org/10.1186/s41239-024-00490-1>
- Mustapa, N. R. N., & Mustapa, S. H. N. (2023). Design and development of mobile application for academic reminder system. *Journal of Computing Research and Innovation (JCRINN)*, 5(4). <https://doi.org/10.24191/jcrinn.v5i4.167>
- Okoye, K. R., & Edokpolor, J. E. (2021). Effect of industrial work experience in developing Technical and Vocational Education undergraduates' employability skills. *Asian Journal of Assessment in Teaching and Learning*, 11(1), 1–12. <https://doi.org/10.37134/ajatel.vol11.1.1.2021>
- Osowski, C. P., Mugisha, R., Bälter, K., & Bälter, O. (2023). Effect of personalized email-based reminders on participants' timeliness in an online education program: Randomized controlled trial. *JMIR Formative Research*, 7, e43977. <https://doi.org/10.2196/43977>
- Pari-Ayala, L. A., & Ramos, H. (2025). Hybrid Web Architecture with AI and Mobile Notifications to Optimize Incident Management in the Public Sector. *Preprints*. <https://doi.org/10.20944/preprints202512.1056.v1>
- Peláez-Sánchez, I. C., Velarde-Camaqui, D., & Glasserman-Morales, L. D. (2024). The impact of large language models on higher education: Exploring the connection between AI and Education 4.0. *Frontiers in Education*, 9. <https://doi.org/10.3389/feduc.2024.1392091>
- Petcharat, W., & Sutthirak, T. (2024). Enhancing the efficiency of internship management through the implementation of progressive web app. *International Journal of Interactive Mobile Technologies*, 18(22), 19–34. <https://doi.org/10.3991/ijim.v18i22.49227>

- Purba, S. (2026). Implementation of role-based access control (RBAC) in a drug stock management information system. *Sistemasi: Jurnal Sistem Informasi*, 15(1). <https://sistemasi.ftik.unisi.ac.id/index.php/stmsi/article/view/5958>
- Qabajeh, M. M., Jaber, D. E.-A., & Mashni, T. K. (2025). Web application for internship management using Laravel framework. *ACIT* 2025. <https://doi.org/10.1109/ACIT68900.2025.11510564>
- Seman, K., Suhaimi, S. A., & Nor, A. M. (2022). Digital Internship Logbook (DIL) for business students in UNITEN. *International Journal of Business Management*, 5(1). <https://journal.uniten.edu.my/index.php/ijbm/article/view/275>
- Setiaji, P., Adiyono, S., Prayogo, A., & Kurniasari, N. (2025). Developing a cloud-native internship management platform: Enhanced efficiency and integration through object-oriented architecture. *Journal of Information Systems and Informatics*, 7(1), 631–648. <https://doi.org/10.51519/journalisi.v7i1.1036>
- Sharma, S., Khare, S., Uniyal, V., & Verma, S. (2022). Hybrid development in Flutter and its widgets. *IEEE ICCR 2022*. <https://doi.org/10.1109/ICCR56254.2022.9995973>
- Sidik, R., Husnaini, A. N., Kholil, M., et al. (2024). Development of a daily monitoring information system for industrial internship. *INTI Journal (Nusa Mandiri)*, 19(2), 1–10. <https://doi.org/10.33480/inti.v19i2.6406>
- Surahmat, A., & Darmawan, R. (2024). Application of SDLC method and Laravel framework in developing web-based information system. *Scientific Journal of Information System*, 2(1), 20–25. <https://doi.org/10.70429/SJIS.V2I1.101>
- Thammastitkul, A. (2023). Cloud-based computing adoption in internship management system for university education. *Naresuan University Journal: Science and Technology*, 31(1), 58–73. <https://doi.org/10.14456/nujst.2023.7>
- Wan Abdul Rahman, W. F., Mohamad Bustamam, M. S., & Putra, Y. H. (2024). The development of an integrated cloud-based system to enhance internship management. *Journal of ICT in Education*, 11(2), 92–110. <https://doi.org/10.37134/jictie.vol11.2.8.2024>
- Weber, K., Cai, T., & Elliston, Z. (2023). I can see clearly now: Using ePortfolios to support reflection and engaged learning in a summer internship program. *International Journal of ePortfolio*, 13(1), 55–67. <https://files.eric.ed.gov/fulltext/EJ1407265.pdf>
- Wibowo, S., Faizah, N. M., & Nurcahyo, W. (2024). Android-based industrial internship management system using Flutter framework. *Journal Mobile Technologies (JMT)*, 2(1), 1–9. <https://doi.org/10.59431/jms.v2i1.292>